

vLLM-Omni: 面向任意到任意 (Any-to-Any) 多模态模型的全解耦服务系统

Peiqi Yin, Jiangyun Zhu, Han Gao, Chenguang Zheng, Yongxiang Huang, Taichang Zhou, Ruirui Yang, Weizhi Liu, Weiqing Chen, Canlin Guo, Didan Deng, Zifeng Mo, Cong Wang, James Cheng, Roger Wang, Hongsheng Liu (通讯作者, 华为)

¹ 华为 AI 框架与数据技术实验室 ² 香港中文大学 ³ 中国科学院软件研究所 ⁴ 中山大学 ⁵ 独立研究者
预印本, 2026 年 2 月 3 日 (arXiv:2602.02204v1)

摘要

能够联合处理文本、图像、视频和音频的“任意到任意” (any-to-any) 多模态模型代表了多模态 AI 的重大进步。然而, 这类模型复杂的架构 (通常将多个自回归 LLM、扩散 Transformer 以及其他专用组件组合在一起) 给高效模型服务带来了巨大挑战。现有的服务系统大多针对单一生成范式定制, 例如用于文本生成的自回归 LLM, 或用于视觉生成的扩散 Transformer, 缺乏对涉及多个互联模型组件的 any-to-any 流水线的支持。因此, 开发者不得不手工处理跨阶段交互, 导致严重的性能下降。我们提出 vLLM-Omni——一个面向 any-to-any 模型的全解耦服务系统。vLLM-Omni 具有新颖的阶段 (stage) 抽象, 使用户能够将复杂的 any-to-any 架构分解为以图 (graph) 形式表示的互联阶段; 并配备解耦的阶段执行后端, 用于优化跨阶段的资源利用率和吞吐量。每个阶段由 LLM 引擎或扩散引擎独立服务, 支持按阶段请求批处理、灵活的 GPU 分配以及统一的阶段间连接器用于数据路由。实验结果表明, 与基线方法相比, vLLM-Omni 将作业完成时间 (JCT) 最多降低了 91.4%。代码已开源: <https://github.com/vllm-project/vllm-omni>。

图 1. Any-to-any 多模态模型架构。(多模态编码器如 Whisper/ViT → 自回归 LLM → 模态生成器如 DiT → 文本/图像/音频/视频输出)

1. 引言

传统大语言模型 (LLM) 在语言理解和推理任务上取得了卓越表现, 例如问答、摘要和代码生成。然而, LLM 局限于纯文本模态, 尤其是在输出端。处理多模态数据的需求日益增长, 推动了多模态模型的发展——通过为 LLM 扩展多种模态的编码器和解码器。其中一个关键趋势是 any-to-any 多模态模型的兴起: 这类统一架构通过端到端训练, 能够无缝地理解和生成文本、图像、视频和音频 (图 1)。与分离的理解与生成流水线相比, 这种统一化实现了更灵活的跨模态推理与交互。近期的 any-to-any 模型在多模态理解任务 (如图像描述、视觉问答、语音助手) 以及生成任务 (如图像编辑、语音翻译、文本转语音合成) 上都达到了 SOTA 性能。

这些 any-to-any 模式的出现使得模型结构比传统 LLM 复杂得多。例如, 为支持音频生成, Qwen-Omni 等现代 any-to-any 模型采用 Thinker-Talker 架构, 将两个自回归 (AR) LLM 连接起来: 一个负责生成文本 token, 另一个负责生成音频 token。为支持图像生成, GLM-Image 等模型通常先用 AR LLM 理解输入, 再连接扩散 Transformer (DiT) 进行视觉合成。更先进的 any-to-any 模型进一步集成多个 AR 和 DiT 组件, 在统一流水线中同时支持音频和视觉输出。

这种模型结构的复杂性给高效服务带来了巨大挑战。现有服务框架通常专用于单一生成范式: vLLM、SGLang 等 LLM 服务框架针对自回归解码优化、仅面向文本生成; 扩散服务框架则针对 DiT 去噪优化、面向图像和视频生成。因此, 这些框架缺乏对 any-to-any 流水线的原生支持——这类流水线涉及多个自回归 LLM、DiT 模型或其他专用神经组件, 且它们必须以定制方式交互以产生多模态输出。开发者无法利用这些框架, 只能诉诸与特定模型紧耦合的定制实现,

导致性能差、可扩展性有限。

为解决这些挑战，我们提出 **vLLM-Omni——面向 any-to-any 模型的全解耦服务系统**，包含阶段抽象前端和阶段执行后端。不同于仅运行单一 AR 解码或 DiT 去噪阶段的现有 LLM 服务框架，vLLM-Omni 通过引入“阶段图”（stage graph）的概念来支持复杂的 any-to-any 架构。借助阶段抽象，用户可以将 any-to-any 模型分解为多个阶段，并显式地将流水线定义为阶段图：节点表示模型阶段（如 AR 或 DiT），边对应用户定义的函数，用于变换中间数据并将其路由到后续阶段。

在给定阶段图规约的情况下，vLLM-Omni 通过解耦的阶段执行后端实现高效服务，优化跨阶段的执行与资源利用。每个阶段由专用执行引擎独立服务——LLM 阶段使用 vLLM，DiT 阶段使用专用扩散引擎。每个引擎执行按阶段的请求批处理以最大化资源利用率。用户可以根据各阶段的计算特性，灵活地为每个阶段分配计算加速器和内存资源。为支持整条流水线的解耦执行，vLLM-Omni 在阶段之间采用统一连接器，实现灵活、可定制的中间数据传输。评估表明，vLLM-Omni 在多种多模态模型上都取得了显著的性能提升：对于 Qwen3-Omni，与基线方法相比，vLLM-Omni 将作业完成时间最多降低了 91.4%。

我们的贡献总结如下：

- 我们提出 vLLM-Omni，一个面向 any-to-any 多模态模型的全解耦服务系统。vLLM-Omni 提出阶段图抽象，实现了对多阶段模型流水线的原生支持。
- 我们提出阶段执行后端，支持按阶段优化、用于数据传输的统一连接器，以及用于视觉生成的扩散引擎。
- 实验证明 vLLM-Omni 在多种 any-to-any 模型和任务上始终优于基线。

2. 背景与动机

2.1 Any-to-Any 多模态模型

Any-to-any

多模态模型通过处理和生成跨多种模态（文本、图像、视频、音频）的输出，扩展了纯文本 LLM 的能力。与标准 LLM 不同，any-to-any 模型采用专用架构进行跨模态理解与生成。在多模态理解方面，现代模型采用专用编码器（音频编码器如 Whisper、Audio Transformer；视觉编码器如 ViT、SigLIP），将多模态输入映射到与 LLM 主干相连的共享嵌入空间。在多模态生成方面，LLM 主干生成嵌入输出，并解析到特定模态的解码器——包括文本转语音模型和图像/视频生成模型——以产生多样化的输出模态¹。这些组件的组合形成了日益复杂的架构，可能结合多个自回归（AR）、扩散 Transformer（DiT）或其他专用生成器。

¹ 本文将具有多种输入和输出模态的模型称为“any-to-any”模型。注意并非所有 any-to-any 模型都支持全部输入输出模态：有些模型支持多模态输入（如文本、图像、音频），但只生成文本和图像，或文本和音频。

多个 AR LLM 解码器。一些 any-to-any 模型在其流水线中使用了多个自回归（AR）LLM 解码器。Qwen-Omni 系列是这一设计的典范（图 2(a)），支持文本、图像、视频和音频输入，同时生成文本和音频输出。该模型由用于输入处理的多模态编码器、负责文本生成的“Thinker”LLM、负责音频 codec 生成的“Talker”LLM，以及用于音频波形重建的“Vocoder”模块组成²。这种 Thinker-Talker 设计在执行流水线中包含了两个串行的自回归 LLM。Ming-Omni 等其他 any-to-any 模型也采用了类似架构。

² 对于 Vocoder，Qwen2.5-Omni 采用 DiT 架构，而 Qwen3-Omni 使用轻量级 CNN 方案。

AR 与专用生成器的组合。许多 any-to-any 模型采用模块化流水线，将 (i) 语义生成与 (ii) 特定模态合成分离。常见的实例化方式是将 AR LLM

与扩散 Transformer (DiT) 结合，将高层语义转化为高保真输出。GLM-Image 遵循这种混合设计 (图 2(b))：首先使用基于 VAE 编码器的 semantic-VQ 提取视觉特征，然后用 9B 自回归 LLM (GLM-4) 进行语义理解和 token 生成，生成的 token 随后由 7B 单流 DiT 解码器消费，合成最终图像。这种"AR + 专用生成器"原则也出现在其他具有视觉或音频输出的 any-to-any 模型中。例如，LongCat-Flash-Omni 使用 560B 参数的 MoE LLM 主干进行自回归 token 生成，随后用轻量级 LSTM/CNN 音频解码器实时重建波形；Step-Audio 使用 130B 参数 LLM 生成语音 token，随后用 DiT flow-matching 混合解码器生成 Mel 频谱、再经神经声码器生成波形。BAGEL 同样可以从这种模块化视角解读：其 Mixture-of-Transformers (MoT) 设计通过不同专家分离多模态语义理解与视觉生成，可视为统一模型中的两个阶段 (图 2(c))。

图 2. 现有 any-to-any 模型的架构。(a) Qwen2.5-Omni; (b) GLM-Image; (c) BAGEL。

2.2 现有 LLM 服务框架面临的挑战

现有开源 LLM 服务框架 (如 vLLM、SGLang) 围绕"以步骤为中心" (step-centric) 的范式设计，针对纯文本 LLM 推理优化。这些框架将迭代逻辑和注意力键值 (KV) 缓存管理封装在运行时中，模型开发者只需通过 forward 函数实现单次前向传播。这种抽象专为顺序文本生成定制——模型从固定输入提示迭代地生成 token 直到输出文本。

然而，any-to-any 多模态 LLM 的出现带来了架构层面的挑战，暴露了 step-centric 设计的局限。any-to-any 模型通常包含多个不同类型的模型组件——如自回归 (AR) LLM、扩散 Transformer (DiT) 以及其他神经网络架构——它们连接成复杂的多阶段流水线。step-centric 抽象出现了根本性的错配：它只能表示文本生成的单次前向传播，无法表达多个异构阶段之间的协调执行与数据流。因此，vLLM 和 SGLang 等现有框架无法支持多模态生成，因为它们的抽象无法表达多阶段流水线。

Qwen-Omni 模型采用包含三个模型组件的 Thinker-Talker 架构，其流水线逻辑无法在 step-centric 框架中表达。开发者需要先为每个阶段独立实现 step-centric 的前向传播，然后在服务框架之外手工编排阶段间传输。工作流程如下：多模态输入通过端到端的 generate() 函数传给 LLM Thinker 阶段，执行编码和 AR 解码循环以生成输出文本；完成后，提取输出隐藏状态并转换为 Talker 阶段的输入嵌入；Talker 通过定制的 generate() 函数执行自己的 AR 生成循环；最后，Talker 阶段的输出传给 Vocoder 阶段进行波形重建。

这种手工实现带来显著的性能损失。第一，多模态生成服务无法利用成熟服务框架提供的效率优化。现有服务系统的输入输出类型固定，甚至难以部署其中的某一个模型阶段；连续批处理 (continuous batching)、分块预填充 (chunked prefill) 等性能优化技术都无法应用。第二，由于模型组件作为单体程序一起实现和执行，计算资源无法在各阶段间高效分配，整条流水线无法被分解或动态调整。阶段的共置 (co-location) 阻碍了细粒度的资源分配，进一步降低服务性能。

3. vLLM-Omni 的设计

vLLM-Omni 是一个面向 any-to-any 多模态模型的解耦服务系统，支持跨异构模型组件的高效、可扩展推理。本节介绍其设计：§3.1 系统架构概览；§3.2 面向 any-to-any 模型编程的阶段抽象接口；§3.3 阶段执行流水线与扩散模型集成；§3.4 阶段解耦的数据传输机制；§3.5 硬件支持。

3.1 概览

图 3 展示了 vLLM-Omni 的总体架构。图 3(a) 为后端架构：一个编排器（orchestrator）管理各阶段的执行并调度 incoming 请求。每个阶段由独立的执行引擎服务，从而实现独立的阶段扩缩容、资源分配和阶段内请求批处理。推理过程中，模型运行器（model runner）迭代地取出批处理请求，对每个被调度的请求应用相应的 preprocess 函数，并在每次迭代执行一次批处理前向步骤。随后，统一连接器在阶段之间传输中间数据，实现整条流水线的完全解耦执行。

图 3(b) 展示了暴露给模型开发者的阶段抽象：any-to-any 模型的每个组件（如 LLM 和 DiT）被实现为一个独立的阶段，配备定制的 preprocess 函数和批处理 forward 函数。preprocess 函数使开发者能够用前序阶段产生的额外数据修改阶段输入。从终端用户的视角看（图 3(c)），vLLM-Omni 暴露运行时配置——包括各阶段的并行策略和内存预算——允许用户在不修改模型代码的情况下调整性能与资源使用。

图 3. vLLM-Omni 架构。(a) 后端架构（编排器、执行引擎、调度器、KV 管理器、数据存储）；(b) 模型开发者视角的阶段模板；(c) 用户视角的运行时配置。

3.2 阶段抽象

vLLM-Omni 为 any-to-any 模型编程提供了灵活易用的前端接口，如图 3(b) 中的模板所示。在该系统中，用户将 any-to-any 模型定义为阶段图：节点表示模型阶段，边表示阶段转移函数。这使得复杂架构（自回归 LLM、DiT 或 CNN 模块）能够被分解为独立的阶段³。具体而言，对于每个 AR 阶段，用户实现一个 preprocess 函数来修改阶段输入，并以与现有 LLM 服务系统相同的 step-centric 方式实现 forward 函数，从而在阶段内实现批处理执行。为管理阶段间的数据流，用户定义阶段转移函数，控制查询状态和中间数据在阶段转换时如何变换。通过组合这些阶段执行与转移定义，阶段图封装了 any-to-any 模型的完整执行流水线。

³ 多模态编码器既可作为独立阶段，也可作为 LLM 阶段的一部分。

图 4 给出了用 vLLM-Omni 实现 Qwen2.5-Omni 模型的示例。如 §2.1 所述，Qwen2.5-Omni 包含三个阶段：(i) 用于文本生成的 LLM Thinker；(ii) 用于音频码生成的 LLM Talker；(iii) 用于波形合成的 DiT Vocoder。模型接收多模态输入，专用编码器将音频、图像和视频转换为嵌入，并与文本输入拼接。这些输入首先送入 Thinker LLM，产生文本输出及相应的隐藏状态。接着，Talker 阶段接收 Thinker 的输出，将 Thinker 隐藏状态和多模态嵌入与 Talker 输入嵌入拼接；Talker LLM 随后自回归地生成 codec token，并在每个解码步骤重复拼接 Thinker 隐藏状态。最后，生成的 codec 序列传给 Vocoder，通过 DiT 去噪产生音频波形。

在 vLLM-Omni 的阶段范式下，用户实现三类函数：(i) 每个模型阶段的 forward 函数（如 thinker_forward、talker_forward、dit_decode）；(ii) 构造阶段输入的 preprocess 函数（如 mm_encode 用于获取多模态嵌入并与 Thinker 输入拼接⁴；process_input 用于拼接 Thinker 隐藏状态与 Talker 输入嵌入，在每次解码迭代中调用）；(iii) 阶段间的转移函数（如 Thinker2Talker 和 Talker2Vocoder，仅调用一次）。典型用法中，用户为每个节点定义 forward 和 preprocess 逻辑，构建阶段图，并为边指定转移函数。通过这种方式，vLLM-Omni 将 any-to-any 模型解耦为模块化阶段，同时充分利用底层服务引擎的性能优化，使用户无需手工处理批处理或调度逻辑即可获得高效的资源利用。

⁴ 在本例中，我们遵循 vLLM 的实现，将多模态编码器视为 Thinker 阶段的一部分。

图 4. Qwen2.5-Omni 的实现示例（模型的阶段图与 workflow）。Qwen3-Omni 类似。各阶段功能：mm_encode——编码多模态嵌入，加入 Thinker 输入；thinker_forward——[批处理] 生成 Thinker 输出（文本）；process_input——拼接 Thinker 隐藏状态、多模态嵌入与 Talker 输入嵌入；talker_forward——[批处理] 从嵌入生成 codec；dit_decode——[批处理] 从码生成波形。

3.3 阶段执行

给定阶段图和用户指定的运行时配置，vLLM-Omni 首先初始化一组执行引擎：每个引擎托管单个模型组件、加载相应模型参数，并按配置的并行策略和内存预算开始服务。随后启动一个编排器进程，管理跨阶段的请求路由和数据交换。

由于每个阶段运行在独立引擎上，vLLM-Omni 可以自然地将复杂 any-to-any 模型结构的执行在各阶段之间解耦。引擎可以根据底层模型阶段的特性和需求配置不同的参数与加速器资源。例如在三阶段的 Qwen3-Omni 流水线中，Thinker 模型最大 (30B)，可以为 Thinker 阶段分配更多加速器内存；Talker 模型较小但计算更密集，可以分配较少内存但更高的并行度和更多加速器。每个引擎还可以启用标准服务优化，如分块预填充 (chunked prefill) 和运行时执行图编译，继承 LLM 服务系统的性能优势。

AR 阶段支持。我们使用 vLLM 引擎服务 AR 阶段。对每个引擎，批处理调度、KV 缓存管理和模型执行由其自己的调度器、KV 管理器和模型运行器独立处理。模型运行器实现了为每个请求执行定制 preprocess 函数的逻辑，从而支持多阶段模型的灵活组合。具体地，我们引入了一个预定义字典用于存储按请求的中间数据，用户可以在转移函数和 preprocess 函数中访问和更新它。preprocess 函数在每次迭代都会被调用，因为某些阶段需要在每个解码步骤将前序阶段的输出与当前前向输入组合（如 Qwen-Omni 的 Talker 阶段）。输出处理器负责应用转移函数，将结果数据存入 CPU 内存，然后传输到托管下一阶段设备。

DiT 阶段支持。vLLM-Omni 将专用扩散引擎无缝集成到其多阶段流水线架构中⁵。通过将扩散过程视为阶段图中的一个独立节点，系统将其核心的解耦服务原则扩展到扩散工作流，确保音频、图像和视频生成任务的高效推理。为最大化吞吐量并降低延迟，该引擎实现了一整套优化技术，包括先进的注意力机制 (flash attention、SAGE attention、TurboAttention)、面向迭代去噪过程的缓存策略 (Tea Cache、cache-dit)，以及并行化方法，如基于 RingAttention 的上下文并行和 Ulysses 序列并行。这些优化使 vLLM-Omni 能够以优于基线实现的吞吐量和延迟服务扩散模型。

⁵ 也可以在该引擎内实现其他生成阶段。

最终，扩散引擎使 vLLM-Omni 能够支持广泛的 SOTA 扩散模型，包括文本到图像生成 (Z-Image、Qwen-Image、Flux)、图像编辑工具 (Qwen-Image-Edit、LongCat-Image-Edit)，以及视频生成变体 (Wan2.2 系列、HunyuanVideo)。

流式阶段输出。在多阶段流水线执行中，某些下游阶段不需要前序阶段的完整输出即可开始计算。例如在 Qwen-Omni 流水线中，只要 Talker 产出初始 token，Vocoder 就可以开始处理音频生成，而不必等待整个序列完成。为支持这种模式，vLLM-Omni 实现了流式阶段输出：中间结果在可用时增量地传输给下游阶段。输出处理器在上游阶段继续执行的同时，异步地将部分输出（如新生成的 token 或嵌入）流式传给下一阶段。通过实现跨阶段的重叠执行，流式阶段输出降低了最终输出的首 token 时间 (TTFT)，并支持向用户流式返回响应，而无需各阶段紧密同步。

3.4 解耦数据传输

vLLM-Omni 通过统一连接器 (unified connector) 接口支持解耦数据传输，将传输层与模型逻辑解耦。受 vLLM 用于 prefill-decode 分离的 KV 缓存传输机制启发，vLLM-Omni 将连接器接口泛化以处理更广泛的数据对象，包括嵌入、隐藏状态以及音频或图像张量。这一统一连接器层负责阶段间的数据移动，实现编码器、prefill、decode 和特定模态生成器等组件的完全解耦。

统一连接器负责在模型阶段之间传输数据。对于单机部署，它通过对小负载使用内联控制队列、对较大负载使用系统共享内存来提供低延迟传输。在分布式多机环境中，我们利用 Ray 编排跨节点执行；基于 Mooncake 的连接器的连接器作为补充，支持基于 TCP 或 RDMA

的传输，允许不同服务器上的阶段通过通用的 put/get 接口交换数据，控制面只传递轻量级元数据。通过将阶段执行与数据传输分离，并允许按边（per-edge）配置连接器，vLLM-Omni 灵活支持异构部署拓扑，并在不改变编程模型的情况下将 any-to-any 流水线扩展到多节点。

统一连接器还处理阶段内传输，包括 prefill 与 decode 之间的 KV 缓存，以及编码器与 prefill 之间的多模态（MM）缓存。该设计与现有的 EPD（encode-prefill-decode）解耦保持兼容。

图 5. 基于统一连接器的解耦数据传输。阶段间传输：Thinker2Talker 隐藏状态、Talker2Vocoder token；阶段内传输：Encode2Prefill 的 MM 缓存、Prefill2Decode 的 KV 缓存。

3.5 硬件支持

vLLM-Omni 支持多样化的硬件平台，以实现灵活的 any-to-any 模型服务。基于 vLLM 的硬件插件架构，vLLM-Omni 通过解耦的插件机制实现跨平台兼容，允许独立注册特定硬件的实现。

4. 实验评估

4.1 实验设置

模型。vLLM-Omni 将 vLLM 的服务能力扩展到具有多模态输出的 any-to-any 任务。我们使用一组有代表性的 SOTA 模型评估系统：(i) Thinker-Talker 架构——在执行流水线中连接两个自回归（AR）模型，即 Qwen3-Omni 和 Qwen2.5-Omni，它们是具有文本和音频输出的 any-to-any 模型；(ii) 两阶段模型——将 AR LLM 与额外的特定模态组件耦合用于生成：如 BAGEL 采用 Mixture-of-Transformer-Experts 设计，理解和生成使用分离的专家（并配备分离的视觉编码器）；MiMo-Audio 则结合 patch 编码器、AR LLM 主干和 patch 解码器来自回归地生成音频 token；(iii) 扩散模型——具有图像或视频输出，即主要基于扩散 Transformer 的 Qwen-Image、Qwen-Image-Edit 和 Wan2.2 系列。

基线系统。对于 Qwen-Omni 模型，由于 vLLM 和 SGLang 只支持其 Thinker 部分，我们使用其默认的 Hugging Face Transformers 实现来评估离线推理性能。对于 BAGEL 和 MiMo-Audio，采用其原始实现作为基线。对于基于扩散的模型，采用 Diffusers 库作为基线。

指标。对于具有音频输出的模型（Qwen-Omni 系列和 MiMo-Audio），我们主要评估实时因子（RTF）和作业完成时间（JCT）。RTF 定义为端到端处理时间与生成音频时长之比。JCT 衡量每个请求从提交到完成的端到端延迟。对于 Qwen-Omni 模型，我们还报告 Thinker 和 Talker 组件的每秒 token 数（TPS）：Thinker TPS 表示每秒生成文本 token 的吞吐量，Talker TPS 表示每秒生成音频 token 的吞吐量。对于视觉生成模型，我们以 JCT 作为主要性能指标。

测试平台。实验在配备两个加速器设备（各 80GB 显存）、24 个 CPU 核心和 192GB 系统内存的服务器上进行。环境为虚拟化配置，使用 vLLM 0.12.0 版本。

4.2 端到端性能

Thinker-Talker 架构。图 6 展示了 vLLM-Omni 与基线在 Qwen-Omni 系列上的端到端性能。我们分别使用 librispeech_asr、food101 和 ucf101-subset 数据集作为音频、图像和视频输入，所有评估均使用每个数据集的前 100 条查询。实验在 2 张 80GB 加速器上运行：基线使用 Transformers 实现的默认张量并行配置；vLLM-Omni 将 Thinker 以张量并行部署在两张加速器上，Talker 放在设备 1，Vocoder 放在设备 0。

对于 Qwen2.5-Omni，与基线 Transformers 实现相比，vLLM-Omni 将 RTF 降低 61.4%，JCT 降低 61.6%。对于 Qwen3-Omni，vLLM-Omni 将 RTF 降低 90.7%，JCT 降低 91.4%。这些结果表明 vLLM-Omni 相比现有实现带来了显著的端到端性能提升。为进一步分析瓶颈，我们报告了 Thinker TPS

和 Talker TPS: vLLM-Omni 在 Qwen2.5-Omni 上分别实现了 1.29× 和 1.97× 的 Thinker 与 Talker TPS 提升; 在 Qwen3-Omni 上实现了 12.97× 的 Thinker TPS 和 7.98× 的 Talker TPS 提升——Thinker 加速超过 10 倍。Qwen3-Omni 上的巨大提升归因于 vLLM-Omni 中实现的额外优化, 而基线实现没有充分利用执行图编译等现代 LLM 服务技术。由于 Qwen3-Omni 的 Thinker 模型 (30B) 远大于 Qwen2.5-Omni (7B), vLLM-Omni 能更好地摊销其优化的执行流水线, 从而获得更高的相对收益。

图 6. Qwen-Omni 模型上的端到端结果 (RTF、JCT、Thinker TPS、Talker TPS, 音频/图像/视频三种输入)。

图 7 展示了 Qwen3-Omni 模型在不同阶段的耗时分解。结果表明, 对两个系统而言, Talker 阶段都占据了总体延迟的大部分, 因为它需要生成的音频 token 远多于 Thinker 生成的文本 token。例如在视频输入任务上, 平均输入 token 数 (含视频 token) 为 841.6, 平均输出文本 token 数为 150.9, 而平均输出音频 token 数达到 545.4。因此, Talker 运行的解码迭代次数远多于 Thinker, 导致更长的延迟。

图 7. Qwen3-Omni 的执行时间分解 (Thinker / Talker / Code2Wav 三阶段占比)。

BAGEL 模型。我们在一张 80GB 加速器上使用 VBench 评估 BAGEL。对于 1024×1024 分辨率的图像生成任务, 基线实现的 JCT 为: 文生图 (T2I) 23.12 秒, 图生图 (I2I) 41.39 秒。我们的方法将 JCT 降至 T2I 9.64 秒、I2I 11.12 秒, 分别实现 2.40× 和 3.72× 的加速。

MiMo-Audio 模型。我们在一张加速器上使用 SeedTTS (文本转语音) 评估 MiMo-Audio。基线实现的 RTF 为 1.39, 而我们的方法在不启用执行图编译时将 RTF 降至 0.60, 启用图编译后降至 0.12, 实现了对基线 11.58× 的加速。

4.3 微观实验

扩散引擎。我们使用 VBench 数据集, 在基于 DiT 的图像和视频生成模型上将 vLLM-Omni 与 Diffusers 进行比较。文生图和图生图分别采用 Qwen-Image 和 Qwen-Image-Edit; 视频生成采用 Wan2.2-T2V 和 Wan2.2-I2V。输出分辨率为 Qwen-Image 1024×1024、Wan2.2 480×640 (80 帧)。结果表明, vLLM-Omni 始终优于 Diffusers, 总体加速比为 1.26×。这一性能提升源于 vLLM-Omni 的扩散引擎复用了 vLLM 的算子优化和 flash-attention 后端, 从而在多样化的生成任务上实现高效执行。

图 8. 基于 DiT 模型的端到端结果 (Qwen-Image 系列 T2I/I2I, Wan2.2 系列 T2V/I2V)。

统一连接器。我们在表 1 中评估了 vLLM-Omni 统一连接器的数据传输开销。结果表明, 相对于整体推理延迟 (数十秒量级), 连接器开销可以忽略不计, 使其成为解耦执行的实用方案。尽管性能开销极小, 统一连接器通过抽象异构部署拓扑间的数据移动, 提供了巨大的灵活性。

延迟 (ms)	Thinker2Talker	Talker2Vocoder
共享内存 (Shared Memory)	5.49	0.53
Mooncake	8.28	3.34

表 1. 使用 vLLM-Omni 统一连接器的数据传输时间 (模型为 Qwen2.5-Omni)。

5. 相关工作

LLM 服务系统。现有的自回归 LLM 服务系统 (如 vLLM、SGLang) 为纯文本或多模态输入、纯文本输出的 LLM 提供高效支持。这些系统提供按步骤的前端接口, 将服务优化对用户屏蔽; 执行后端集成了 LLM 执行优化, 包括注意力实现 (如 paged attention、flash attention)、KV 缓存管理和前缀树缓存, 并纳入分块预填充、连续批处理、Prefill-Decode (PD) 分离等优化以实现更低延迟和更高吞吐量, 还支

持数据并行、张量并行等多种并行策略。由于 vLLM-Omni 的执行引擎扩展自 vLLM 引擎，它继承了这些针对 any-to-any 流水线中 AR 阶段的优化。

多模态模型服务。当前的 LLM 服务系统支持多模态输入的 LLM，具有 Encode-Prefill-Decode (EPD) 分离和多模态嵌入缓存等优化，但仍聚焦于纯文本输出场景，缺乏对多模态输出模型的支持。另一方面，基于扩散的系统通过量化注意力、并行去噪和缓存策略等优化，高效加速视觉和音频生成；然而这些框架专用于扩散模型，在处理复杂架构时力不从心，尤其是在集成重量级 LLM 文本编码器时。相比之下，vLLM-Omni 通过将自回归模型与扩散模型无缝结合，为复杂流水线提供统一支持，实现 any-to-any 多模态模型的高效服务。这种能力对于大规模部署下一代 any-to-any 模型至关重要。

6. 结论

本文提出了 vLLM-Omni，一个用于高效部署 any-to-any 多模态模型的服务系统。我们工作的核心洞见是：将复杂的 any-to-any 模型架构分解为阶段图，其中每个阶段都可以被独立优化和执行。通过解耦的阶段执行后端，vLLM-Omni 实现了对多样化 any-to-any 模型的高效服务支持。实验结果表明，与现有方法相比，vLLM-Omni 取得了显著的性能提升。

(参考文献部分从略)